

An Introduction to Language Models ¹

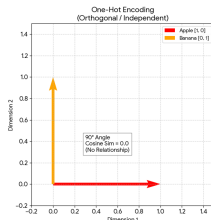
Shaobo Li

University of Kansas

¹This lecture slides was created in collaboration with Google Gemini.

The Fundamental Problem

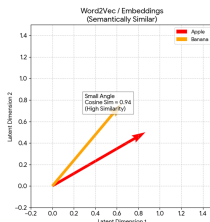
- Neural Networks require numerical input ($X_1, X_2, \dots$)
- Language is symbolic (Words, Characters)
- Old Approach: **One-Hot Encoding** (dummy coding)
 - Apple = $[1, 0, 0, \dots, 0]$
 - Banana = $[0, 1, 0, \dots, 0]$
- **Issue:** Orthogonal vectors (independent)



How do we teach the machine that apples and bananas are similar?

Word Embeddings (Word2Vec)

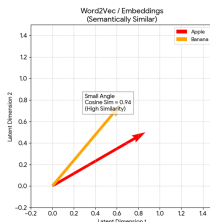
- Map words to dense vectors in a continuous vector space.
 - e.g., Apple = $[3.6, 0.9, -1.4, \dots, 1.7]$ instead of $[1, 0, 0, \dots, 0]$
- We can compute Semantic Similarity: (using *Cosine Similarity*)



- **Analogy Properties:**
 - a famous example: "King" – "Man" + "Woman" $\approx$ "Queen"

Word Embeddings (Word2Vec)

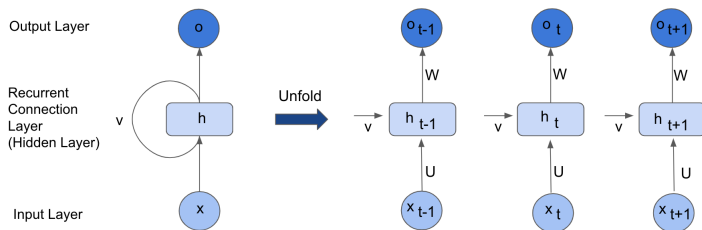
- Map words to dense vectors in a continuous vector space.
 - e.g., Apple = $[3.6, 0.9, -1.4, \dots, 1.7]$ instead of $[1, 0, 0, \dots, 0]$
- We can compute Semantic Similarity: (using *Cosine Similarity*)



- **Analogy Properties:**
 - a famous example: "King" – "Man" + "Woman" $\approx$ "Queen"

Now let's work on an example in Colab. [example 1]; [example 2]

Recurrent Neural Networks (RNN)



- Language is sequential, not static.
- The "Hidden State" (h_t) acting as memory.

$$h_t = \sigma(W_h h_{t-1} + W_x x_t)$$

- Passes information from previous word to current word.

The Problem with RNNs

- **Vanishing Gradient Problem**

- As we backpropagate through time, gradients shrink.
- The network "forgets" the beginning of the sentence.

- **Example:** "I grew up in **France**... [200 words] ... I speak fluent _____."

- RNNs fail to connect "France" to "French" over long distances.
- We need a better memory system.

Long Short-Term Memory (LSTM)

- Introduces a **Cell State** (The Highway).
- Uses **Gates** to control information flow:
 - 1 **Forget Gate**: What to throw away?
 - 2 **Input Gate**: What to store?
 - 3 **Output Gate**: What to pass to the next step?
- Solved the long-term dependency problem (standard until 2017).

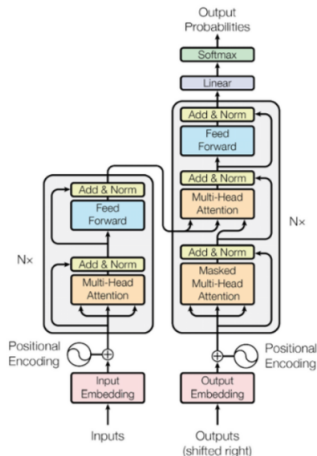
Now let's work on an example in Colab. [click here to open](#)

The Transformer Revolution

- 2017 Paper: *"Attention Is All You Need"* (Google).

Transformer

Attention Is All You Need



The Transformer Revolution

- Drop the recurrence (loops).
- Process the **entire sentence** at once (Parallelization).
- Use **Self-Attention**.

Self-Attention Mechanism

- Every word looks at every other word to calculate relevance.
- Example: "The **bank** of the **river**."
- The model calculates a score (dot product) between "Bank" and "River".
- **The Math:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

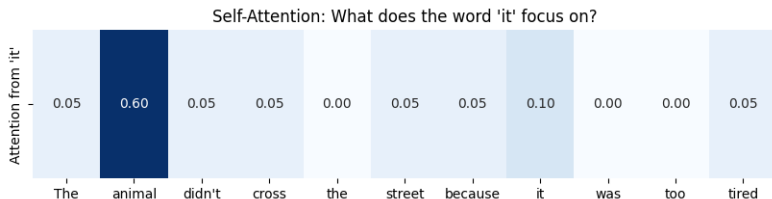
- Query (Q): What are the things I am looking for?
- Key (K): What are the things that I have?
- Value (V): What are the things that I will communicate?
- High score = High Attention = Context is merged.

Now let's look at an example in Colab.

[a simple demo]; [an algorithm demo]

Visualizing Attention

- "The animal didn't cross the street because **it** was too tired."
- The word **it** attends strongly to **animal**.



- If we change "tired" to "narrow", **it** attends to **street**.

Here is a great [visual explanation of transformer](#).

Transformers

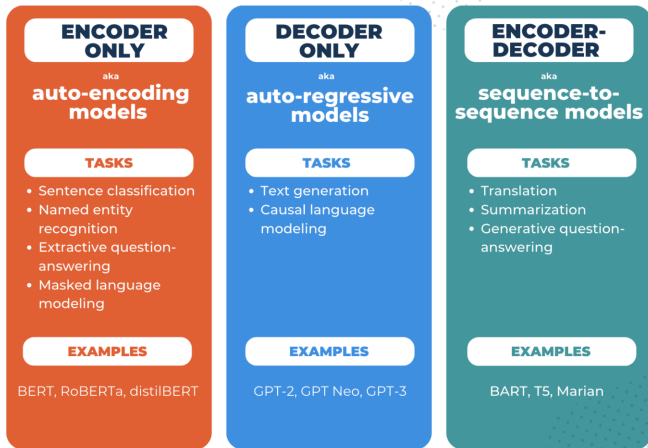


Image from [here](#)

Summary: The Evolution

- 1 **Embeddings (Word2Vec):** Words become vectors. Meaning is distance.
- 2 **RNNs:** Added sequence processing (Context).
- 3 **LSTMs:** Fixed memory issues (Longer Context).
- 4 **Transformers:** Parallel processing + Infinite context window via Attention.

This architecture (Transformer) is the "T" in ChatGPT.

Next? Multi-modal Transformer (processing and understanding text, images, audio simultaneously).